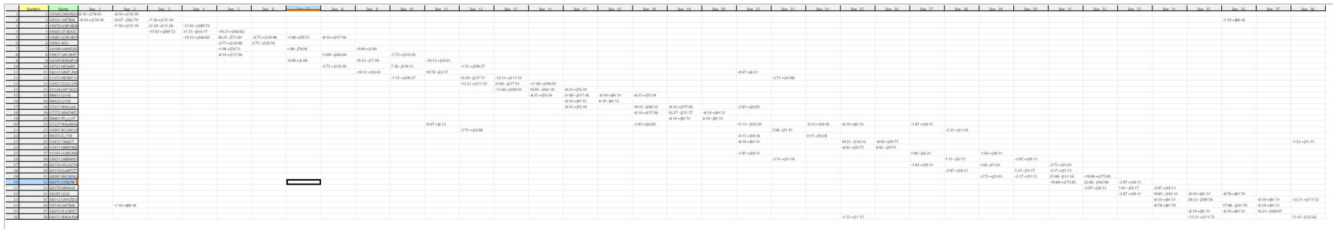
EE 530 Power Systems & Machines Fundamentals – HW 2

1. Access the Ybus and Jacobian matrices of the JKM SEWA PowerWorld Simulator transmission system model. What is the percentage of non-zero matrix entries?

Using the equation from class, the percentage of non-zero entries is 9.10%
Buses: 36
Transmission lines: 41
Transformers: 2
Non-zero elements $= n + 2N_{\text{T-lines}} = 36 + 2(41) = 118$
Percentage of non-zero entries $= \frac{118}{36^2} = 9.10\%$

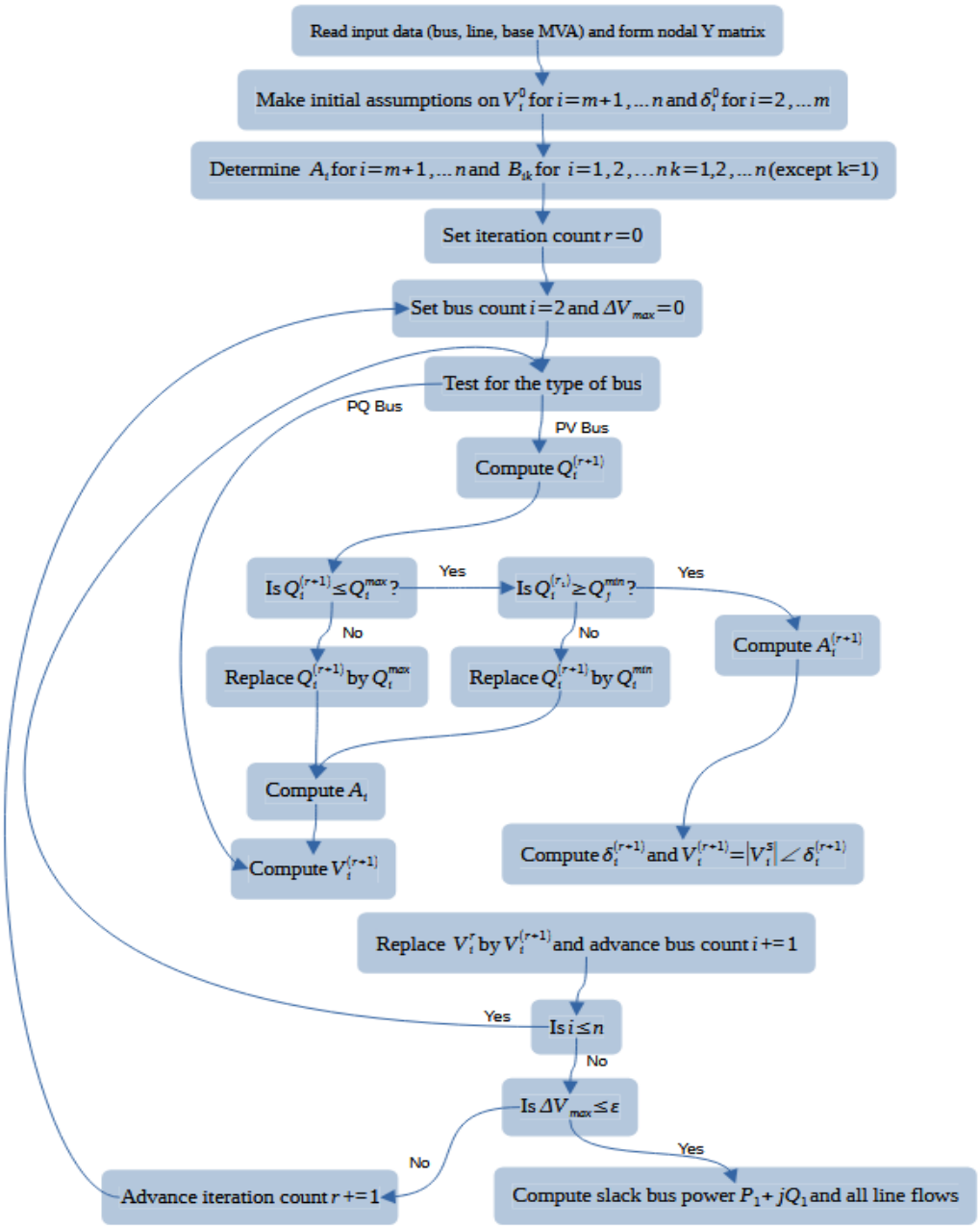
However, if you count the cells in Powerworld, the percentage of non-zero y-bus matrix entries is **8.95%**.



2. Design Process Flow Diagrams (PFDs) that clearly outline the steps involved for both the Gauss-Seidel and Newton-Raphson power-flow solution processes
   a) Write a brief introductory description of each power-flow solution process. Clearly label each step within the PFDs. Provide brief descriptions of the purpose and function of each step. Use standard PFD symbols and shapes, which can be found within software tools such as Draw.io, Miro, or MS Viso.

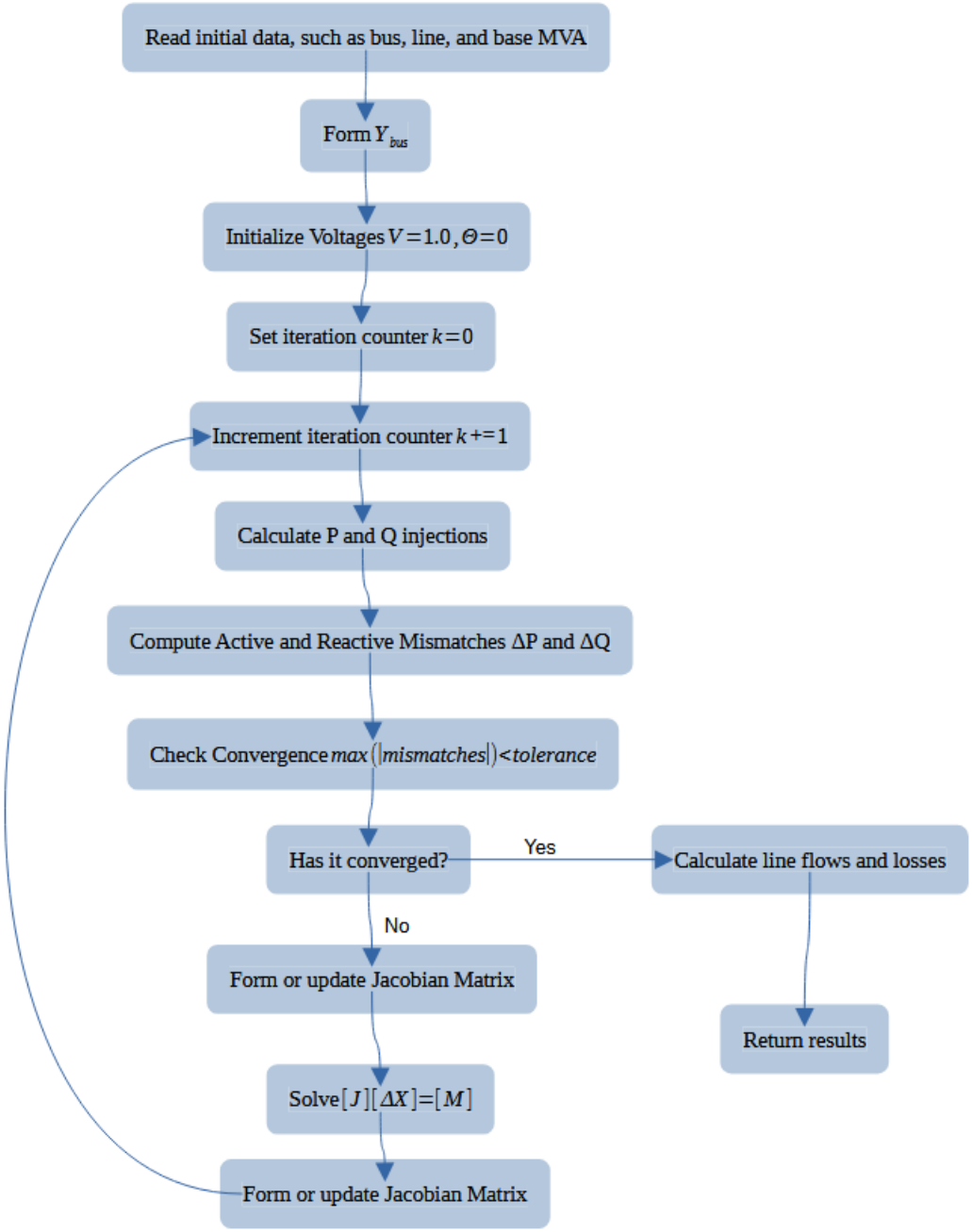
The Gauss-Seidel (G-S) method is an iterative algorithm that solves non-linear power-flow equations by updating bus voltages sequentially, utilizing the most recently calculated values for each subsequent bus. The Newton-Raphson (N-R) method utilizes a Taylor series expansion and a Jacobian matrix of partial derivatives to compute voltage magnitude and angle corrections simultaneously, resulting in highly efficient quadratic convergence.

Gauss-Siedel Method:



https://www.eeeguide.com/gauss-seidel-method/ and
https://www.sciencedirect.com/topics/engineering/gauss-seidel-method used as reference

Newton-Raphson Method:



b) Write a pseudocode in a structured, readable manner using English-like language that describes the process of the N-R method

```
BEGIN Newton-Raphson Power Flow
    READ system bus, transmission line, and base power data
    CONSTRUCT Ybus admittance matrix
    INITIALIZE V = 1.0 and Theta = 0.0 for all PQ buses
    SET iteration_count = 0

    WHILE iteration_count < max_iterations DO
        CALCULATE scheduled P and Q for all buses
        CALCULATE active (delta_P) and reactive (delta_Q) power mismatches

        IF maximum(mismatch) < tolerance THEN
            BREAK loop (Convergence Achieved)
        END IF

        CONSTRUCT Jacobian matrix using partial derivatives of power equations
        SOLVE linear system [Jacobian] * [delta_Theta; delta_V] = [delta_P; delta_Q]
        UPDATE bus voltages: V_new = V_old + delta_V, Theta_new = Theta_old +
        delta_Theta
        INCREMENT iteration_count
    END WHILE

    CALCULATE line flows and system losses
END
```

3. Consider Chapman's power_flow.m power systems analysis code.
   a) What iterative computational technique is employed in the code?

Gauss-Seidel iteration with an acceleration/relaxation factor

b) Modify Chapman's code such that three varieties of load may be modeled; in addition to PQ, allow for modeling of constant Z and constant I load

See the attached python file power_flow.py. Code is also at https://github.com/LemonTorrent/EE530-HW2-Powerflow

c) Consider the data input files FiveBus_PQ and FiveBus_PQZI
   i. 'Bus types' modified and added: LP, LZ, LI
   ii. Pload and Qload input assuming 1.0 bus voltage for all load types. Turn in only the modified portions of the code, highlighting changes and additions made.

Summary of changes of code are shown in file changes.txt attached or in repo https://github.com/LemonTorrent/EE530-HW2-Powerflow, but I've also pasted some of the updated logic below:

```python
for i in range(n_bus):
    btype = normalize_bus_type(bus[i]['type'])
```

```
if btype == 'SL':
    continue  # skip slack

# Dynamic load modeling based on bus voltage magnitude
v_mag = abs(Vbus[i])
if btype == 'LZ':
    # Constant Impedance: P and Q proportional to |V|^2
    bus[i]['PL'] = bus[i]['PL_nominal'] * (v_mag ** 2)
    bus[i]['QL'] = bus[i]['QL_nominal'] * (v_mag ** 2)
elif btype == 'LI':
    # Constant Current: P and Q proportional to |V|
    bus[i]['PL'] = bus[i]['PL_nominal'] * v_mag
    bus[i]['QL'] = bus[i]['QL_nominal'] * v_mag
else:
    # Constant Power (PQ / LP or PV): P and Q constant
    bus[i]['PL'] = bus[i]['PL_nominal']
    bus[i]['QL'] = bus[i]['QL_nominal']

# Update net P and Q injections
bus[i]['P'] = bus[i]['PG'] - bus[i]['PL']
if btype != 'PV':
    bus[i]['Q'] = bus[i]['QG'] - bus[i]['QL'] + bus[i]['QC']

# If PV bus, update Q estimate from current voltages
if btype == 'PV':
    temp = 0+0j
    for j in range(n_bus):
        temp += ybus[i][j] * Vbus[j]
    temp = Vbus[i].conjugate() * temp
    bus[i]['Q'] = -temp.imag
```

4. Consider the sparse matrix techniques discussed in the lecture. Write a script that determines the vectors DIAG, OFFDIAG, COL, and ROW for an N x N matrix

See attached file titled sparse_matrix_vector.py. File is also in github repo
https://github.com/LemonTorrent/EE530-HW2-Powerflow